

Souvenirs

Amaru está a comprar lembranças numa loja estrangeira. Existem N **tipos** de lembranças. Há um número infinito de lembranças de cada tipo disponíveis na loja.

Cada tipo de lembrança tem um preço fixo. Nomeadamente, uma lembrança do tipo i ($0 \leq i < N$) tem um preço de $P[i]$ moedas, em que $P[i]$ é um número inteiro positivo.

Amaru sabe que os tipos de lembranças estão ordenados por ordem decrescente de preço e que os preços das lembranças são distintos. Mais especificamente, $P[0] > P[1] > \dots > P[N-1] > 0$. Além disso, ele conseguiu saber o valor de $P[0]$. Infelizmente, Amaru não tem mais nenhuma informação sobre os preços das lembranças.

Para comprar algumas lembranças, Amaru efectua uma série de transações com o vendedor.

Cada transação consiste nas seguintes etapas:

1. Amaru entrega um número (positivo) de moedas ao vendedor.
2. O vendedor coloca essas moedas numa pilha sobre a mesa na sala dos fundos, onde Amaru não as pode ver.
3. O vendedor considera cada tipo de lembrança $0, 1, \dots, N-1$ por esta ordem, uma a uma. Cada tipo é considerado **exatamente uma vez** por transação.
 - Ao considerar o tipo de lembrança i , se o número atual de moedas no monte for pelo menos $P[i]$, então
 - o vendedor retira $P[i]$ moedas do monte e
 - o vendedor coloca uma lembrança do tipo i na mesa.
4. O vendedor dá ao Amaru todas as moedas restantes na pilha e todas as lembranças que estão na mesa.

Nota que não há moedas ou lembranças na mesa antes do início de uma transação.

A tua tarefa consiste em dar instruções ao Amaru para efetuar um determinado número de transações, de modo a que:

- em cada transação ele compre **pelo menos uma** lembrança e
 - no total, ele compre **exatamente** i lembranças do tipo i , para cada i tal que $0 \leq i < N$.
- Nota que isto significa que Amaru não deve comprar nenhuma lembrança do tipo 0.

Amaru não tem de minimizar o número de transações e dispõe de uma quantidade ilimitada de moedas.

Detalhes de Implementação

Deves implementar a seguinte função.

```
void buy_souvenirs(int N, long long P0)
```

- N : o número de tipos de lembranças.
- $P0$: o valor de $P[0]$.
- Esta função é chamada exatamente uma vez para cada caso de teste.

A função acima pode fazer chamadas à seguinte função para instruir o Amaru a realizar uma transação:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : o número de moedas entregues ao vendedor pelo Amaru.
- O procedimento devolve um par. O primeiro elemento do par é um vector L , que contém os tipos de lembranças que foram compradas (por ordem crescente). O segundo elemento é um número inteiro R , o número de moedas devolvidas ao Amaru após a transação.
- É necessário que $P[0] > M \geq P[N - 1]$. A condição $P[0] > M$ garante que Amaru não compra nenhuma lembrança do tipo 0, e $M \geq P[N - 1]$ garante que Amaru compra pelo menos uma lembrança. Se estas condições não forem cumpridas, a tua solução receberá o veredito `Output isn't correct: Invalid argument`. Nota que, ao contrário de $P[0]$, o valor de $P[N - 1]$ não é fornecido no input.
- A função pode ser chamada no máximo 5000 vezes em cada caso de teste.

O comportamento do avaliador é **não adaptativo**. Isto significa que a sequência de preços P é fixada antes de `buy_souvenirs` ser chamada.

Restrições

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ para cada i tal que $0 \leq i < N$.
- $P[i] > P[i + 1]$ para cada i tal que $0 \leq i < N - 1$.

Subtarefas e Pontuação

Subtarefa	Pontos	Restrições Adicionais
1	4	$N = 2$
2	3	$P[i] = N - i$ para cada i tal que $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ para cada i tal que $0 \leq i < N$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ para cada i tal que $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ para cada i tal que $0 \leq i < N - 1$.
6	33	Nenhuma restrição adicional.

Exemplo

Considera a seguinte chamada.

```
buy_souvenirs(3, 4)
```

Existem $N = 3$ tipos de lembranças e $P[0] = 4$. Observa que só há três sequências possíveis de preços P : $[4, 3, 2]$, $[4, 3, 1]$, e $[4, 2, 1]$.

Supõe que `buy_souvenirs` chama `transaction(2)`. Supõe que a chamada devolve $([2], 1)$, o que significa que Amaru comprou uma lembrança do tipo 2 e o vendedor devolveu-lhe 1 moeda. Observa que isso nos permite deduzir que $P = [4, 3, 1]$, pois:

- Para $P = [4, 3, 2]$, `transaction(2)` teria retornado $([2], 0)$.
- Para $P = [4, 2, 1]$, `transaction(2)` teria retornado $([1], 0)$.

Então `buy_souvenirs` pode chamar `transaction(3)`, que devolve $([1], 0)$, o que significa que Amaru comprou uma lembrança do tipo 1 e o vendedor devolveu-lhe 0 moedas. Até agora, no total, ele comprou uma lembrança do tipo 1 e uma lembrança do tipo 2.

Finalmente, `buy_souvenirs` pode chamar `transaction(1)`, que devolve $([2], 0)$, o que significa que Amaru comprou uma lembrança do tipo 2. Nota que também poderíamos ter usado `transaction(2)` aqui. Nesta altura, no total Amaru tem uma lembrança do tipo 1 e duas lembranças do tipo 2, como pedido.

Avaliador Exemplo

Formato do input:

N

$P[0] \quad P[1] \quad \dots \quad P[N-1]$

Formato do output:

$Q[0] \quad Q[1] \quad \dots \quad Q[N-1]$

Aqui $Q[i]$ é o número de lembranças do tipo i compradas no total para cada i tal que $0 \leq i < N$.